```
try
{
    some_promise.set_value(calculate_value());
}
catch(...)
{
    some_promise.set_exception(std::current_exception());
}
```

This uses `std::current_exception()` to retrieve the thrown exception; the alternative here would be to use `std::copy_exception()` to store a new exception directly without throwing:

```
some_promise.set_exception(std::copy_exception(std::logic_error("foo ")));
```

This is much cleaner than using a `try`/`catch` block if the type of the exception is known, and it should be used in preference; not only does it simplify the code, but it also provides the compiler with greater opportunity to optimize the code.

Another way to store an exception in a future is to destroy the `std::promise` or `std::packaged_task` associated with the future without calling either of the set functions on the promise or invoking the packaged task. In either case, the destructor of the `std::promise` or `std::packaged_task` will store a `std::future_error` exception with an error code of `std::future_errc::broken_promise` in the associated state if the future isn't already *ready*; by creating a future you make a promise to provide a value or exception, and by destroying the source of that value or exception without providing one, you break that promise. If the compiler didn't store anything in the future in this case, waiting threads could potentially wait forever.

Up until now all the examples have used `std::future`. However, `std::future` has its limitations, not the least of which being that only one thread can wait for the result. If you need to wait for the same event from more than one thread, you need to use `std::shared_future` instead.

4.2.5. Waiting from multiple threads

Although `std::future` handles all the synchronization necessary to transfer data from one thread to another, calls to the member functions of a particular `std::future` instance are not synchronized with each other. If you access a single `std::future` object from multiple threads without additional synchronization, you have a *data race* and undefined behavior. This is by design: `std::future` models unique ownership of the asynchronous result, and the one-shot nature of `get()` makes such concurrent access pointless anyway—only one thread can retrieve the value, because after the first call to `get()` there's no value left to retrieve.

If your fabulous design for your concurrent code requires that multiple threads can wait for the same event, don't despair just yet; `std::shared_future` allows exactly that. Whereas `std::future` is only *moveable*, so ownership can be transferred between instances, but only one instance refers to a particular asynchronous result at a time, `std::shared_future` instances are *copyable*, so you can have multiple objects referring to the same associated state.